

A PROJECT INTERNSHIP
ON
FAKE CURRENCY DETECTION USING CNN

Submitted in Partial fulfillment of the requirements for the award of the degree in

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

BY

A.ABHINAYA SRI (22NN1A1262)

MD. MUSARRAT NAZIA (22NN1A1288)

N. GOWTHAMI (22NN1A1292)

V.LAKSHMI SRILATHA (22NN1A12B8)

Under the Esteemed Guidance of

Dr.V.PAVANI

Associate Professor



DEPARTMENT OF INFORMATION TECHNOLOGY

**VIGNAN'S NIRULA INSTITUTE OF TECHNOLOGY AND
SCIENCE FOR WOMEN**

(Approved by A.I.C.T.E, New Delhi and Affiliated to J.N.T.U.K, Kakinada)

PEDAPALAKALURU, GUNTUR-525005

(2022-2026)

CERTIFICATE

This is to certify that the project work entitled “**Fake currency Detection using CNN**” is a bonafide work of submitted by A. Abhinaya Sri (22NN1A1262), MD. Musarrat Nazia(22NN1A12288), N. Gowthami(22NN1A1292), V. Lakshmi Srilatha(22NN1A12B8) the Department of Information Technology in the partial fulfillment forward of degree of BACHELOR OF TECHNOLOGY in Information Technology in VIGNAN’S NIRULA INSTITUTE OF TECHNOLOGY AND SCIENCE FOR WOMEN, GUNTUR.

Project Guide

Dr. V. Pavani

Associate Professor

Head of the Department

Dr. K. V. S. S. Rama Krishna

Associate Professor

External Examiner

DECLARATION

We hereby declare that the work described in the project work, entitled “**Fake Currency Detection using CNN**” which is submitted by us in partial fulfilment for the award of **Bachelor of Technology** in the department of **Information Technology** to the **Vignan’s Nirula Institute of Technology and Science for Women**, affiliated to **Jawaharlal Nehru Technological University Kakinada, Andhra Pradesh**, is the result of work done by us under the guidance of Dr. V. Pavani.

The work is original and has not been submitted for any Degree/Diploma of this or any other University.

Place: Guntur

A. Abhinaya Sri (22NN1A1262)

Md. Musarrat Nazia (22NN1A1288)

N. Gowthami (22NN1A1292)

V. Lakshmi Srilatha (22NN1A12B8)

ACKNOWLEDGEMENT

We are profoundly grateful to express our deep sense of gratitude and respect towards our honorable chairman, **Dr. Lavu Rathaiah** sir, chairman of Vignan's Group for his precious support in the college.

We express our heartfelt gratitude to our beloved principal **Dr. P. Radhika** for giving a chance to study in our esteemed institution and providing us all the required resources.

We would like to thank **Dr. K. V. S. S. Rama Krishna** Associate Professor, Head of the Department, Information Technology, for his extended and continuous support, valuable guidance and timely advice in the completion of this project thesis.

We wish to express our profound sense of sincere gratitude to our project guide **Dr. V. Pavani** Associate Professor, Department of Information Technology, without whose help, guidance, and motivation this project thesis could not have been completed successfully.

We also thank all the faculty members of the Department of Information Technology for their help and guidance on numerous occasions, which has given us the cogency to build-up adamant aspiration over the completion of our project thesis.

Finally, We thank one and all who directly or indirectly helped us to complete our project thesis successfully.

A. Abhinaya Sri (22NN1A1262)

Md. Musarrat Nazia (22NN1A1288)

N. Gowthami (22NN1A1292)

V. Lakshmi Srilatha (22NN1A12B8)

FAKE CURRENCY DETECTION USING CONVOLUTIONAL NEURAL NETWORK

TABLE OF CONTENTS

CONTENT	PAGE NO.
1. INTRODUCTION	01-03
2. LITERATURE SURVEY	04-07
3. SOFTWARE ENVIRONMENT	08-13
4. EXISTING MODELS	14-15
5. PROPOSED SYSTEM	16-23
6. OUTPUT IMAGES	24-26
7. CONCLUSION	27-28
8. REFERENCES	29-31
9. CERTIFICATES	32-36

ABSTRACT

The fabrication of counterfeit currency is seen as a serious offense since it is created without the government's consent. Large-scale counterfeit currency note printing has expanded due to advancements in color printing technology. A simple laser printer can now print a currency note with maximum accuracy, whereas in the past, printing could only be done at a print house. As a result, the problem of counterfeit notes rather than real ones has significantly increased. It is the main issue that many nations, including India, are dealing with. Even though banks and other big businesses have automated tools to identify counterfeit currency, the typical consumer finds it extremely difficult to tell the difference.

In this study, deep convolutional neural networks (CNNs) are used to detect counterfeit Indian banknotes in a novel way. The suggested method makes use of datasets of both authentic and fake Indian currency. Several sources of high-resolution photos of authentic and fake banknotes were gathered to produce this dataset. To reduce noise and enhance quality, images have undergone pre-processing. A deep VGG16 model was then trained using the previously processed photos. In order to differentiate real banknotes from counterfeit ones, the model is intended to learn their features. The trained model's performance was assessed using a number of criteria after it was tested on distinct datasets of authentic and fake currency. The results of the study demonstrate that the suggested method may accurately and successfully identify fake Indian currency. The Deep VGG16 model outperforms current methods in detecting counterfeit currency, with over 99% accuracy. All things considered, the suggested method is a trustworthy and effective way to identify fake Indian currency and lessen the financial losses brought on by its widespread use.

CHAPTER 1

INTRODUCTION

INTRODUCTION

To address the challenges faced by many people and our universities in distinguishing between counterfeit and real currencies, a fake currency detection is proposed. The project involves making use of Digital Image Processing Domain - Digital image processing is the use of computer algorithms to perform image processing on digital images. With the help of technology, the banking sector is also getting modern day by day. Undoubtedly, this places a serious necessity of automatic fake currency detection in automated teller machine and automatic goods seller machine. This has prompted many researchers to work on developing robust and efficient automatic counter currency machine.

Self-service machines equipped with the capacity to recognize notes have now found their place in modern day vending machines such as candy, soft drinks, bus or railway tickets. The basic premise of currency recognition technology is identifying and extracting both visible and invisible characteristics of currency notes. So far a number of techniques have been developed for the identification of the currency note. But the best way to identify a particular banknote is by observing its physical features like color and size. However, this approach is useless if the note is smudged or frayed. If a note is heavily dirty, its inherent color will also change. Hence it is crucial that how we derive the features of an image of the currency note and deploy an appropriate algorithms for enhancing the recognition of the note. A simplified algorithm is demonstrated, which has been quite effective. In India, the Reserve Bank of India has the exclusive mandate to issue currency notes. From time to time and like many other central banks round the world, Reserve Bank modifies the pattern of the bank notes.

This model does not require a precise process of feature extraction rather it can be trained on dataset and be made efficient enough to tackle issues of counterfeit currency. In time, this category of systems has potential to grow as well. Technology has given us devices such as computers and mobile phones that have become a springboard for rapid growth. The evolving of technology directly contributed to issues such as the creation of fake currency detection systems as the existing problem of identifying counterfeits still remains relevant. Such projects are based on digital

images, which are furthermore processed using computer algorithms. This development is important on the level of the nation because it helps to prevent corruption that impedes the development of a country.

Operating microscopically and counterfeit detection pens, watermarking and optically variable ink have been some preferred methods adopted to detect fake currency over the years. To improve its detection improved use of a computer interface and digital image processing techniques is offered which shows how application systems can be used to authenticate a note. The proposed system uses Simple Neural Network classifiers resulting in providing an effective and easy to use solution for fake currency detection.

CHAPTER 2

LITERATURE SURVEY

LITERATURE SURVEY

S. Arya et al [1] have observed a continuous rise in the circulation of counterfeit currency notes. To address this issue, they propose an effective and innovative system designed to detect fake currency. The detection process involves counting the number of interruptions in a thread line; if the count is zero, the note is deemed authentic, while any interruptions indicate a counterfeit. Additionally, the entropy of the currency notes is calculated to enhance the efficiency of fake currency detection.

M. Laavanya et al [2] highlight that advancements in printing and scanning technologies have exacerbated the problem of counterfeiting. This proliferation of counterfeit currency undermines the economy and diminishes the value of legitimate currency. Consequently, the need for effective counterfeit detection has become paramount. Traditional methods predominantly rely on hardware and image processing techniques, which are often inefficient and time-consuming. To address these shortcomings, they propose utilizing a deep convolutional neural network for counterfeit detection. Once the feature map is established, the network can identify fake currency in real-time, efficiently distinguishing between counterfeit notes of denominations such as 2000, 500, 200, and 50 with reduced time expenditure.

A. Vidhate et al [3] note that advancements in color printing technology have significantly increased the production of counterfeit currency notes. In the past, such printing required specialized facilities, but now, individuals can produce highly accurate counterfeit notes using standard laser printers. This has led to a substantial rise in the prevalence of fake notes, overshadowing genuine currency. Although banks and large organizations have implemented automatic machines for detecting counterfeit notes, it remains challenging for the average person to differentiate between authentic and fake currency. This situation has contributed to increased corruption, impeding national progress. In response, they propose the development of an application system to aid in counterfeit detection.

Counterfeit items are defined as imitations of authentic products, a concept articulated by A.A. Mandankandy et al [4]. In the banking sector, counterfeit currency remains a significant threat. Although numerous detection methods exist, the rise of easily accessible image manipulation tools exacerbates the issue. Identifying critical regions within currency notes is essential for evaluation.

Classifiers are employed to determine whether the extracted features are genuine or counterfeit. In the absence of classifiers, one can compare the segmented currency image with the original note's regions; however, this method alone is insufficient for verifying the authenticity of the image. Misalignment and edge discrepancies may lead to counterfeit notes being mistakenly identified as genuine. To enhance accuracy, it is imperative that the extracted features undergo processing by classifiers. This paper also encompasses a comparative study of image segmentation or thresholding, feature extraction, classification, and selection methodologies.

A. Upadhyaya et al [5] conducted an analysis of counterfeit currency detection techniques for classification models in 2018. Their review emphasized the methodologies applied to specific characteristic features, detailing the success rates of each method in detecting counterfeit currency. Additionally, the study examined widely accepted statistical classification techniques for currency authentication. A comparative analysis between Logistic Regression and Linear Discriminant Analysis (LDA) was undertaken to identify the superior model for currency authentication. The findings indicated that the classification model utilizing Logistic Regression achieved a higher accuracy rate of 99% compared to LDA.

T. D. Pham et al [6] highlighted the significance of automatic recognition of counterfeit banknotes in the context of practical banknote handling. Research in this area has primarily focused on methods implemented in automatic sorting machines, which utilize multiple imaging sensors or specialized sensors designed for capturing banknote images under various conditions.

S. T. Kara et al [7] introduced a detection system for counterfeit currency that employs a hybrid methodology combining CNN and ResNet models to achieve precise outcomes. This system leverages convolutional neural networks (CNN) and residual networks (ResNet) to identify counterfeit money by analyzing its dimensions, colors, and serial numbers. The suggested automated system for recognizing Indian currency notes assesses their authenticity by applying CNN techniques.

B. Srilatha et al [8] presented a solution for detecting counterfeit currency that utilizes Convolutional Neural Networks (CNN) and image processing, showcasing an innovative method that incorporates artificial intelligence (AI) and machine learning for effective detection of fakes. The method involves using a transfer-learned CNN, a deep learning architecture that has been

trained on a collection of images of both authentic and counterfeit currency. This CNN grasps the complex features distinguishing real banknotes from fake ones, enabling it to successfully identify counterfeit currency in real-time. By employing transfer learning, the CNN can utilize insights gained from a varied dataset, enhancing its capacity to detect subtle characteristics linked to forged notes.

M. Ghonge et al [9] introduced a method titled "Real Time Fake Note Detection using Deep Convolutional Neural Network." This approach relies on a deep convolutional neural network to identify counterfeit currency via a mobile application. A self-created dataset was utilized to train the model. Additionally, it was evaluated using live images taken with a smartphone camera.

S. Mehta et al[10] introduced a method for "Fake Currency Detection Across Five Classes," which utilizes a CNN-SVM combined approach. This work presents a unified model that employs a Convolutional Neural Network (CNN) to extract features and Support Vector Machines (SVM) for identifying counterfeit currency. The model aims to differentiate between genuine currency and four types of fake notes, encompassing a total of ten different banknotes. The error graph produced during training and validation depicted the learning curve, demonstrating effective performance as the error consistently increased over 200 epochs of iterations.

CHAPTER 3

SOFTWARE ENVIRONMENT

SOFTWARE ENVIRONMENT

INTRODUCTION TO PYTHON

Python is a high-level, interpreted scripting language developed in the late 1980s by Guido van Rossum at the National Research Institute for Mathematics and Computer Science in the Netherlands. The initial version was published at the alt. Sources newsgroup in 1991, and version 1.0 was released in 1994. Python 2.0 was released in 2000, and the 2.x versions were the prevalent releases until December 2008. At that time, the development team made the decision to release version 3.0, which contained a few relatively small but significant changes that were not backward compatible with the 2.x versions. Python 2 and 3 are very similar, and some features of Python 3 have been back ported to Python 2. But in general, they remain not quite compatible. Both Python 2 and 3 have continued to be maintained and developed, with periodic release updates for both. As of this writing, the most recent versions available are 2.7.15 and 3.6.5.

WHY CHOOSE PYTHON?

For some applications that are particularly computationally intensive like graphics processing or intense number, this can be limiting. In practice, however, for most programs, the difference execution speed is measured in milliseconds, or seconds at most, and not appreciably noticeable to a human user. The expediency of coding in an interpreted language is typically worth it for most applications. Python is Free. The Python interpreter is developed under an OSI-approved opensource license making it free to install use and distribute even for commercial purposes.

A version of the interpreter is available for virtually any platform there is, including all flavors of Unix, Windows, macOS, smart phones and tablets, and probably anything else you ever heard. A version even exists for the half dozen people remaining who use OS/2. Python is Portable Because Python code is interpreted and not compiled into native machine instructions, code written for one platform will work on any other platform that has the Python interpreter installed.

Python is Simple

Python-3 has 33 keywords, and Python-2 has 31. By contrast, C++ has 62, Java has 53, and Visual Basic has more than 120, though these latter examples probably vary somewhat by implementation or dialect. Python code has a simple and clean structure that is easy to learn and easy to read. As you will see, the language definition enforces code structure that is easy to read. But It's Not That Simple for all its syntactical simplicity, Python supports most constructs that would be expected in a very high-level language, including complex dynamic data types, structured and functional programming, and object-oriented programming.

Additionally, a very extensive library of classes and functions is available that provides capability well beyond what is built into the language, such as database manipulation. Python accomplishes what many programming languages don't the language itself is simply designed, but it is very versatile in terms of what you can accomplish with it.

WHAT IS ANACONDA?

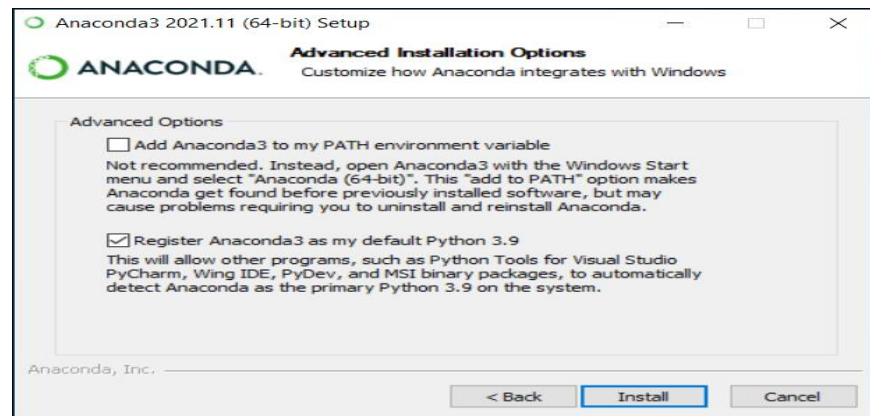
Anaconda is a package manager, an environment manager, and Python distribution that contains a collection of many open source packages. This is advantageous as when you are working on a data science project, you will find that you need many different packages (numpy, scikit-learn, scipy, pandas to name a few), which an installation of Anaconda comes preinstalled with.

If you need additional packages after installing Anaconda, you can use Anaconda's package manager, conda, or pip to install those packages. This is highly advantageous as you don't have to manage dependencies between multiple packages yourself. Conda even makes it easy to switch between Python 2 and 3. In fact, an installation of Anaconda is also

HOW TO INSTALL ANACONDA?

1. Download the Anaconda installer.
2. Go to your Downloads folder and double-click the installer to launch. To prevent permission errors, do not launch the installer from the Favorites folder
3. Click Next.
4. Read the licensing terms and click I Agree.

5. It is recommended that you install for just me, which will install Anaconda Distribution to just the current user account. Only select an install for All users if you need to install for all users' accounts on the computer.
6. Click Next.
7. Select a destination folder to install Anaconda and click Next. Install Anaconda to a directory path that does not contain spaces or unicode characters. For more information on destination folders.



i.

a. Fig1 Anaconda Installation

8. Choose whether to add Anaconda to your PATH environment variable or register Anaconda as your default Python. We don't recommend adding Anaconda to your PATH environment variable, since this can interfere with other software. Unless you plan on installing and running multiple versions of Anaconda or multiple versions of Python, accept the default and leave this box checked. Instead, use Anaconda software by opening Anaconda Navigator or the Anaconda Prompt from the Start Menu

OpenCV

OpenCV (Open Source Computer Vision Library) is an open source computer vision and machine learning software library. OpenCV was built to provide a common infrastructure for computer vision applications and to accelerate the use of machine perception in the commercial products. Being an Apache 2 licensed product, OpenCV makes it easy for businesses to utilize and modify the code. The library has more than 2500 optimized algorithms, which includes a comprehensive set of both classic and state-of-the-art computer vision and machine learning algorithms. These algorithms can be used to detect and recognize faces, identify objects, classify human actions in videos, track camera movements, track moving objects, extract 3D models of objects, produce 3D

point clouds from stereo cameras, stitch images together to produce a high resolution image of an entire scene, find similar images from an image database, remove red eyes from images taken using flash, follow eye movements, recognize scenery and establish markers to overlay it with augmented reality

USES of OpenCV

- Reading an image
- Extracting the RGB values of a pixel
- Extracting the Region of Interest (ROI)
- Resizing the Image
- Rotating the Image
- Drawing a Rectangle
- Displaying text

IMUTILS

A series of convenience functions to make basic image processing functions such as translation, rotation, resizing, skeletonization, displaying Matplotlib images, sorting contours, detecting edges, and much more easier with OpenCV and both Python 2.7 and Python 3.

Numpy

NumPy is a Python library used for working with arrays. It also has functions for working in domain of linear algebra, Fourier transform, and matrices. NumPy was created in 2005 by Travis Oliphant. It is an open source project and you can use it freely. NumPy stands for Numerical Python.

Numpy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays. It is the fundamental package for scientific computing with Python. Besides its obvious scientific uses, numpy can also be used as an efficient multi-dimensional container of generic data.

Matplotlib

Matplotlib is a Python library used for creating static, interactive, and animated visualizations in Python. It provides a wide range of plotting functions to create various types of graphs, charts, histograms, scatter plots, etc. When we plot data on a graph, that graph will be drawn on a figure. By default, a figure is automatically generated during plotting in matplotlib. We can create multiple figures, change the figure's properties, and add subplots to a figure.

Skimage

Scikit-image, or skimage, is an open source Python package designed for image preprocessing. If you have previously worked with sklearn, getting started with skimage will be a piece of cake. Even if you are completely new to Python, skimage is fairly easy to learn and use. This module includes tools to transform images and volumetric data. Geometric transformation: These transforms change the shape or position of an image.

Keras and tensorflow

Keras is a deep learning API written in Python, running on top of the machine learning platform TensorFlow. It was developed with a focus on enabling fast experimentation

- A. Simple -- but not simplistic. Keras reduces developer cognitive load to free you to focus on the parts of the problem that really matter. • Flexible -- Keras adopts the principle of progressive disclosure of complexity: simple workflows should be quick and easy, while arbitrarily advanced workflows should be possible
- B. Powerful -- Keras provides industry-strength performance and scalability TensorFlow is an end-to-end, open-source machine learning platform. You can think of it as an infrastructure layer for differentiable programming. It combines four key abilities:

Efficiently executing low-level tensor operations on CPU, GPU, or TPU.

CHAPTER 4

EXISTING MODELS

EXISTING MODELS

Counterfeit currency poses a significant threat to a nation's economy, thereby affecting global growth. The production of counterfeit money or the creation of fraudulent features on currency is classified as a criminal offense. Such currency-related crimes fall under criminal law and are categorized as economic crimes. In recent years, numerous researchers have suggested various methods for identifying and detecting counterfeit currency. This pressing issue has led to the development of a range of solutions, including hardware-based techniques, image processing, and machine learning approaches. Some of the approaches such as

1. The detection of fake currency notes is achieved by counting the interruptions in the thread line. For confirming whether the note is original or forged, it is based on the interruptions: if interruptions are zero, it is an original note; otherwise, it would be a fake. Most former techniques involve hardware and image-processing techniques. These methods are inefficient as far as time is taken to identify the counterfeit currency.
2. The proposed model dealt with the issue by identifying counterfeits using deep convolution neural networks. The model identifies the currency fakes by reviewing the currency images. Methods like watermarking, optically variable inks, security threads, latent images, techniques like counterfeit detection pens are some of the methods used for detection of fake currencies.
3. This model has an application for fake note detection using image processing. It will detect variations in barcodes among real and fake ones and will tell differences between the image under consideration and the prototype. CNN classifiers will be used to detect fake currency. The proposed app for fake currency detection will be simple, accurate and easy to use.
4. Extraction of important areas in currencies is a basic task and thus those important regions would be evaluated. The model states that classifiers can distinguish the extracted features as either real or counterfeit. Without classifiers, original notes region could be compared with the cut currency image, but that will not help to authenticate photo image. Some important areas are separated and may have the alignment and edges different, such that some fake currency note image(s) may appear to be original.

CHAPTER 5

PROPOSED SYSTEM

PROPOSED SYSTEM

By applying a Deep Learning using Convolutional Neural Network (CNN) significantly improved the effectiveness and detection on currency identification with greater accuracy, faster and more efficient. This is a fully automated technique that needs no human touch and very less complex. The model proposed is based on deep learning and image processing techniques that is capable of classifying currency notes in categories like ₹10, ₹50, ₹100, ₹500, ₹2000. More specifically, the method includes two approaches to detect counterfeit Indian currency with a Simple Neural Network (SNN) and a Convolutional Neural Network (CNN).

Steps in the Proposed Methodology

1.Data Collection and Preprocessing

A. Data Collection

- You require an authentic and diverse collection of various denominations of Indian currency notes, as well as a compilation of fake notes. This causes the model to learn to recognize a diverse variety of features from different notes.
- The training data must cover a multitude of counterfeits: printed copies, digitally altered notes, and forged security features.
- These techniques include augmentation techniques like rotation, flipping, scaling to artificially expand the dataset to enhance generalization of the model.

B. Pre-Processing

- i. Resizing: Used rescaling to resize the size of the images to have all the same size, since they are going to use them at the end in neural network.
- ii. Normalization: This is the process of changing inputs to a neural network to a common scale (for example, scaling pixel values to be between 0 and 1) and helps neural networks to converge better. This adds to minimizing cases like a slow training process or infliction of gradient instability during the entire optimization process.
- iii. Grayscale Conversion: If the images are converted to grayscale, then we can reduce the problem complexity, and we will only focus on structural features like bleed lines, security threads as well as text patterns. Some other functions such as security patterns based on color, you may consider keeping the images in RGB.


```

# Reading the image
test_img = cv2.imread(path)
cv2.imshow("a", test_img)
cv2.waitKey(0)
cv2.destroyAllWindows()

# Pre- processing

# Pre- processing

# Resizing the image
test_img = cv2.resize(test_img, (1167, 519))

# Gaussian Blur
blur_test_img = cv2.GaussianBlur(test_img, (5,5), 0)

# Grayscale conversion
gray_test_image = cv2.cvtColor(blur_test_img, cv2.COLOR_BGR2GRAY)

def preprocessing():
    # Showing original currency note
    plt.imshow(gray_test_image, 'gray')
    plt.title('Input image after pre- processing')
    plt.show()
    progress['value']=5
    ProgressWin.update_idletasks()

    # Updating the progress
    progress['value']=5
    ProgressWin.update_idletasks()

```

Fig1 Data Collection

```

# Importing all necessary libraries
import cv2
# Importing Opencv library for image processing and computer vision
import numpy as np
# Importing numpy library
import matplotlib.pyplot as plt
# Importing matplotlib library to plot the images directly in notebook
from skimage.metrics import structural_similarity as ssim
# Importing ssim calculating modules from skimage library

# Importing tkinter library to build GUI
from tkinter import *
from tkinter.ttk import Progressbar

import time

#Resizing the Plots
plt.rcParams["figure.figsize"] = (12, 12)

# Declaring variable for progress bar to store the total progress
myProgress = 0.0

# This Ipython magic will retrieve the path of the input image which is stored when the gui_1.ipynb is executed.
%store -r path

# A sample path:
#path = r'Dataset\500_s1.jpg'
path='500_dataset\500_s1.jpg'
#path='real.jpg'

print('Path of input image: ', path)
Path of input image: 500_dataset\500_s1.jpg

```

Fig2 Preprocessing

2.Model Architecture

1.SNN

- The SNN is designed to process pairs of images and learn how similar or dissimilar the images are to each other. It's particularly useful in feature comparison tasks such as determining if a particular feature in the currency note matches a template of a genuine feature.

2.CNN

- CNNs are strong in terms of spatial hierarchy analysis of images. Their architecture is perfect for detecting subtle patterns like bleed lines and watermarks in currency.

Architecture

A. Convolutional Layers

- Low-level pattern detectors, edges, corners, texture in image. Going further down, the CNN can learn more advanced patterns such as bleed lines, watermarks, number panels.
- Layers uses convolutional filters (kernels) which move across the image to generate feature maps.

B. Pooling Layers

- After each convolution layer, pooling, such as max pooling is applied to the feature maps to reduce spatial dimensions. Only the most important features are retained in the process.
- This lowers down the complexity of computation and the model pays attention to higher-level patterns.

C. Fully Connected Layers

- All the neurons are interconnected, allowing the network to finally take a decision about the image, either the note is real or fake.
- Fully connected layers classify the images based on aggregated features coming from all previous layers.

3.Training

The training procedure focuses on finding differences in important characteristics like:

A. Left Bleed Lines and Right Bleed Lines:

- Can be identified as important places in the pictures, these lines are exclusive to real cash notes.
- The exact locations of these lines will be determined by ORB by identifying edges and corners where there is a noticeable shift in intensity.

Feature 8: Left Bleed Lines

```

# Function to count number of bleed lines in left side- Feature 8
# Algo 2

def testFeature_8():
    plt.rcParams["figure.figsize"] = (5, 5)

    # Check Feature 8- Left bleed lines
    print('\nANALYSIS OF FEATURE 8 : LEFT BLEED LINES\n')

    # Cropping the region in which left bleed lines are present- Feature extraction
    crop = test_img[120:240, 12:35]

    img = crop.copy()
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    _, thresh = cv2.threshold(gray, 130, 255, cv2.THRESH_BINARY)

    plt.imshow(thresh, 'gray')
    plt.show()

    whitePixelValue = 255 # White pixel
    blackPixelValue = 0 # Black pixel

    width = thresh.shape[1] # width of thresholded image

    # Result will be stored here
    result = [] # will contain the number of black regions in each column (if the column is non- erroneous)
    num_of_cols = 0 # will contain the number of non- erroneous columns

    # Non erroneous columns are those columns which contains less noise.
    print('Number of black regions found in each column: ')

    # iteration over each column in the cropped image
    for j in range(width):
        col = thresh[:, j:j+1] # Extracting each column of thresholded image
        count = 0 # Counter to count number of black regions in each extracted column

        for i in range(len(col)-1):

            # Taking two consecutive pixels and storing their intensity value
            pixel1_value = col[i][0]
            pixel2_value = col[i+1][0]

            #-----
            # This part modifies any error pixels, if present.
            # Actually in a binary threshold, all pixels should be either white or black.
            # If due to some error pixels other than white or black are present, then the pixel is taken as white pixel

            if pixel1_value != 0 and pixel1_value != 255:
                pixel1_value = 255
            if pixel2_value != 0 and pixel2_value != 255:
                pixel2_value = 255

            #-----

            # If current pixel is white and next pixel is black, then increment the counter.
            # This shows that a new black region has been discovered.
            if pixel1_value == whitePixelValue and pixel2_value == blackPixelValue:
                count += 1

            # If the counter is less than 10, it is a valid column. (less noise is present)
            if count > 0 and count < 10:
                print(count)
                result.append(count)
                num_of_cols += 1
            else:
                # discard the count if it is too great e.g. count> 10 (Erroneous Column)
                # This removes/ drops those columns which contain too much noise
                print(count, 'Erroneous -> discarded')

    # Printing necessary details
    print('\nNumber of columns examined: ', width)
    print('Number of non- erroneous columns found: ', num_of_cols)

    if num_of_cols != 0:
        average_count = sum(result)/num_of_cols
    else:
        average_count = -1
        print('Error occurred- Division by 0')

    print('\nAverage number of black regions is: ', average_count)

```

Fig2 Left Bleed Lines Detection

Feature 9: Right Bleed Lines

```

# Function to count number of bleed Lines in right side- Feature 9
def testFeature_9():
    plt.rcParams["figure.figsize"] = (5, 5)

    # Check Feature 9- Right bleed Lines
    print('\nANALYSIS OF FEATURE 9 : RIGHT BLEED LINES\n')

    # Cropping the region in which Left bleed Lines are present- Feature extraction
    crop = test_img[120:260, 1135:1155]

    img = crop.copy()
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Thresholding the image
    _, thresh = cv2.threshold(gray, 130, 255, cv2.THRESH_BINARY)

    plt.imshow(thresh, 'gray')
    plt.show()

    whitePixelValue = 255 # White pixel
    blackPixelValue = 0 # Black pixel

    width = thresh.shape[1] # width of thresholded image

    # Result will be stored here
    result = [] # will contain the number of black regions in each column (if the column is non- erroneous)
    num_of_cols = 0 # will contain the number of non- erroneous columns

    # Non erroneous columns are those columns which contains less noise.

    print('Number of black regions found in each column: ')

    # Iteration over each column in the cropped image
    for j in range(width):
        col = thresh[:, j:j+1] # Extracting each column of thresholded image
        count = 0 # Counter to count number of black regions in each extracted column

        # Iterating over each row (or pixel) in the current column
        for i in range(len(col)-1):
            # Taking two consecutive pixels and storing their intensity value
            pixel1_value = col[i][0]
            pixel2_value = col[i+1][0]

```

Fig 3 Right Bleed Lines

```

    # If current pixel is white and next pixel is black, then increment the counter.
    # This shows that a new black region has been discovered.
    if pixel1_value == whitePixelValue and pixel2_value == blackPixelValue:
        count += 1

    # If the counter is less than 10, it is a valid column. (Less noise is present)
    if count > 0 and count < 10:
        print(count)
        result.append(count)
        num_of_cols += 1
    else:
        # discard the count if it is too great e.g. count > 10 (Erroneous Column)
        # This removes/ drops those columns which contain too much noise
        print(count, 'Erroneous -> discarded')

    # Printing necessary details
    print('\nNumber of columns examined: ', width)
    print('Number of non- erroneous columns found: ', num_of_cols)

    if num_of_cols != 0:
        average_count = sum(result)/num_of_cols
    else:
        average_count = -1
        print('Error occured- Division by 0')

    print('\nAverage number of black regions is: ', average_count)

    # Storing the thresholded image and average number of bleed Lines detected
    global right_BL_result
    right_BL_result = [thresh, average_count]
    global myProgress

    # Updating progress in progress bar
    progress['value']=85
    ProgressWin.update_idletasks()

```

Fig 4 Right Bleed Lines

B. SSIM Scores

- To determine how comparable the structural features are between an input image and a reference image, the model learns to calculate the SSIM between the two.
- The SNN looks for significant differences between the image pairings by comparing them according to SSIM values.

```
# Calculating SSIM of the two images sent as parameters
def calculateSSIM(template_img, query_img):
    min_w = min(template_img.shape[1], query_img.shape[1])
    min_h = min(template_img.shape[0], query_img.shape[0])

    # Resizing the two images so that both have same dimensions
    img1 = cv2.resize(template_img, (min_w, min_h))
    img2 = cv2.resize(query_img, (min_w, min_h))

    # Conversion to gray-scale
    img1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
    img2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

    # Plotting the images
    plt.subplot(1, 2, 1)
    plt.imshow(img1, 'gray')

    plt.subplot(1, 2, 2)
    plt.imshow(img2, 'gray')

    plt.show()

    # Find the SSIM score and return
    score = ssim(img1, img2)
    return score
```

Fig 5 Calculation of SSIM score

C. Currency Number Panels

- ORB can be used to determine the direction, size, and location of the currency number panels, such as the note's digits or serial numbers. These are important aspects of the currency's design that counterfeiters frequently overlook.

Feature 10: Currency Number Panel

```
# Cropping out the number panel.
# Cropping out the number panel. - FEATURE EXTRACTION

crop = gray_test_image[410:500, 700:1080] # This is the cropped gray image
crop_bgr = test_img[410:500, 700:1080] # This is the cropped BGR image

# Algo 3
def testFeature_10():
    plt.imshow(crop_bgr)
    plt.show()

    plt.rcParams["figure.figsize"] = (7, 7)
    print('\nANALYSIS OF FEATURE 10 : NUMBER PANEL \n')

    test_passed = False # If true, then the test is successful
    res_img_list = [] # List of images of successful cases
    count = 0 # Stores number of cases which are successful
    i = 0

    # THRESHOLDING at multiple values
    # Start from 95 as threshold value, increase the threshold value by 5 every time and check if 9 characters are detected in the
    # If 9 characters are detected in at least one of the cases, the currency number panel is verified.
    # If more than 1 cases pass, the best image will be chosen from the successful cases.
    for thresh_value in range(95, 155, 5):
        # Thresholding at current value
        _, thresh = cv2.threshold(crop, thresh_value, 255, cv2.THRESH_BINARY)

        print('----> Threshold ' + str(i+1) + ' with Threshold value ' + str(thresh_value) + ' :')
        i += 1

    copy = crop_bgr.copy()
```

```

# Finding all the contours in the image of the number panel- CONTOUR DETECTION
img = cv2.bitwise_and(crop, crop, mask=thresh)
contours, hierarchy = cv2.findContours(img, cv2.RETR_TREE, cv2.CHAIN_APPROX_NONE)

h_img, w_img = img.shape[:2]

# cv2.drawContours(copy, contours, -1, (0, 0, 255), 1)

# Storing the details of all the BOUNDING RECTANGLES for each contour
bounding_rect_list = []

for contour in contours:
    [x, y, w, h] = cv2.boundingRect(contour)

    if x != 0:
        bounding_rect_list.append([x,y,w,h])

# Sorting the List of rectangles
# Rectangles will get sorted according to the x coordinate of the top Left corner
bounding_rect_list.sort()

# ELIMINATION OF ERRONEOUS RECTANGLES
# Min area is taken as 150
min_area = 150
res_list = []

# Storing all rectangles having area greater than the min_area in a separate List
for i in range(0, len(bounding_rect_list)):
    if i >= len(bounding_rect_list):
        break
    if bounding_rect_list[i][2]*bounding_rect_list[i][3] > min_area:
        res_list.append(bounding_rect_list[i])

# Eliminating the rectangles that are present within a bigger rectangle
i = 0
while i < len(res_list):
    [x,y,w,h] = res_list[i]
    j = i+1
    while j < len(res_list):
        [x0,y0,w0,h0] = res_list[j]

        if (x+w) >= x0+w0:
            res_list.pop(j)
        else:
            break
    i += 1

```

```

# Eliminating unnecessary rectangles
i = 0
while i < len(res_list):
    [x,y,w,h] = res_list[i]

    if (h_img-(y+h)) > 40: # Eliminating the rectangles whose Lower edge is further away from Lower edge of the image
        res_list.pop(i)
    elif h < 17: # Eliminating the rectangles whose height is Less than 17 pixels
        res_list.pop(i)
    else:
        i += 1

for rect in res_list: # Drawing the remaining rectangles
    [x,y,w,h] = rect
    cv2.rectangle(copy, (x, y), (x + w, y + h), (0, 0, 255), 1)

# COUNTING REMAINING RECTANGLES
# result of each image
if len(res_list) == 9: # If number of rectangles detected is greater than 9, test passed
    test_passed = True
    res_img_list.append(copy)
    count += 1
    print('Test Successful: 9 letters found!')
else:
    print('Unsuccessful!')

# If three consecutive cases pass the test, then break
if count == 3:
    break

# Choosing the BEST IMAGE to be displayed
# Even if a single case passes the test, then the currency number panel is verified.
# Selecting the best image to display
if count == 0: # If none of the cases passes the test
    best_img = crop_bgr
elif count == 1: # If 1 case passes the test, then the image used in 1st case is selected as the best image
    best_img = res_img_list[0]
elif count == 2: # If 2 cases pass the test, then the image used in 2nd case is selected as best image
    best_img = res_img_list[1]
else: # If >= 3 cases pass the test, then the image used in 3rd case is selected as best image
    best_img = res_img_list[2]

```

Fig 6,7,8 Currency Number Panel

CHAPTER 6

OUTPUT IMAGES

OUTPUT IMAGES

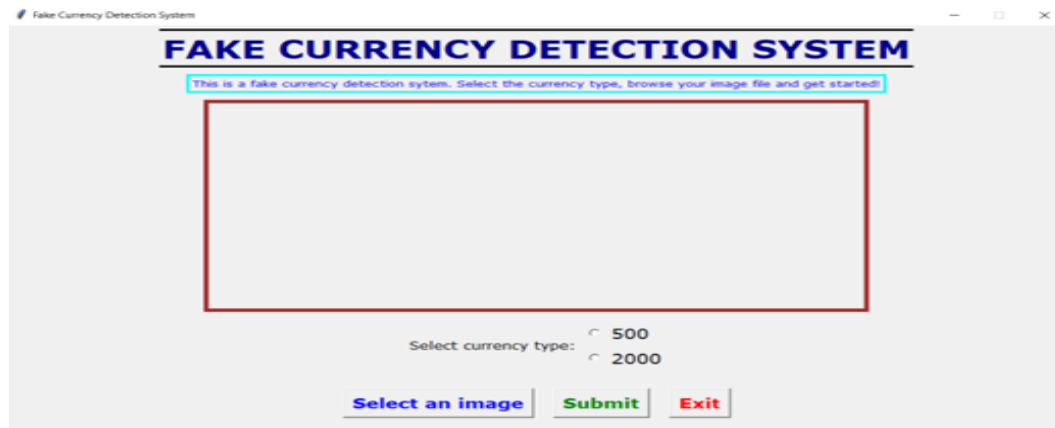


Image 1 Selection of Currency type



Image2 500 currency note selected



Image3 Input Image sent for Pre-processing



Image4 Input Image after Pre Processing

Final Score- set list:

```
Feature 1 : [0.12409424266956555, 0.12409424266956555, 0.12409424266956555, 0.12409424266956555, 0.12409424266956555, 0.12409424266956555]
Feature 2 : [0.09127704172723941, 0.09127704172723941, 0.09127704172723941, 0.09127704172723941, 0.09127704172723941, 0.09127704172723941]
Feature 3 : [0.10537908505531052, 0.10537908505531052, 0.10537908505531052, 0.10537908505531052, 0.10537908505531052, 0.10537908505531052]
Feature 4 : [0.15605686974876684, 0.15605686974876684, 0.15605686974876684, 0.15605686974876684, 0.15605686974876684, 0.15605686974876684]
Feature 5 : [0.10909120102095225, 0.10909120102095225, 0.10909120102095225, 0.10909120102095225, 0.10909120102095225, 0.10909120102095225]
Feature 6 : [0.08991837048942306, 0.08991837048942306, 0.08991837048942306, 0.08991837048942306, 0.08991837048942306, 0.08991837048942306]
Feature 7 : [0.12464783948792084, 0.12464783948792084, 0.12464783948792084, 0.12464783948792084, 0.12464783948792084, 0.12464783948792084]
```

Image 5 final Score Set List

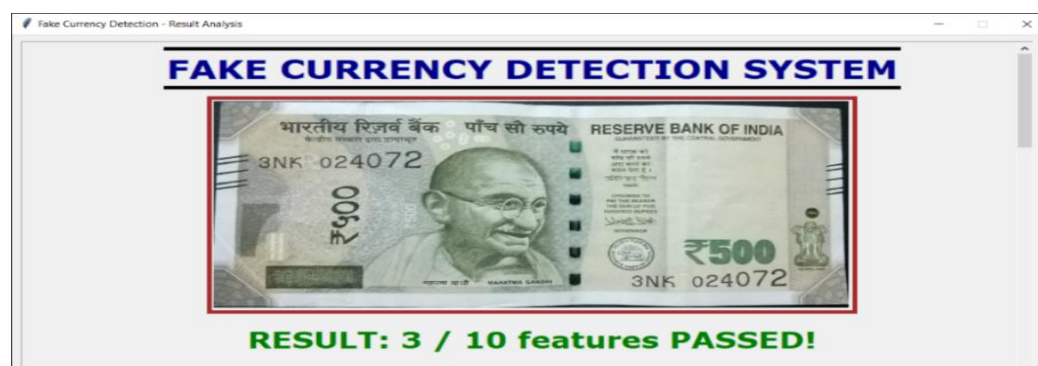


Image 6 Result Analysis

CHAPTER 7

CONCLUSION

CONCLUSION

The project of Fake Currency Detection using CNN intends to offer a better model practically while dealing with the Indian Currency Counterfeit through high end computer vision technique. By integrating Convolutional Neural Networks (CNN) used for feature extraction and classification and SSN used for similarity comparisons, it aims to extract and classify the unique features of bleed lines, currency number panels and structural patterns to ascertain if the currency in question is fake. This methodology seeks to showcase the importance of preprocessing like resizing or cropping, normalization, noise reduction and input scaling so that the model is fed with clean images.

With SSIM (Structural Similarity Index) introduced to test the difference between the original and the reproduced notes which means that even the slightest discrepancy in crucial aspects of the note are detectable whereas this is a task that is usually tough to do with the normal human eye. With the right mix of counterfeit and original notes the CNN models can complement each other's efforts to detect counterfeits in high volumes with accuracy. The strength of the CNN model lies in its ability to classify notes by recognizing their visual patterns.

FUTURE ENHANCEMENT

- Technology is advancing at a rapid pace these days. The proposed technique can be used to detect coins as well as recognize fake currencies.
- Other countries currencies can be added, and a comparison between them can be made.
- When a picture is loaded into the training folder from the outside, it does not provide 100 percent accuracy. By optimizing the system, we can solve this problem.
- Real-Time Detection: Optimize it for real-time counterfeit detection using lightweight variants of CNNs such as MobileNet or Tiny-YOLO for fast processing. Implement it in handheld or mobile devices so it may be used in banking, retail, and other industries.

CHAPTER 8

REFERENCES

REFERENCES

- [1]. A. Singh, K. Bhoyar, A. Pandey, P. Mankani, A. Tekriwal, Detection of fake currency using image processing. *Int. J. Eng. Res. Technol. (IJERT)* 8(12) (2019).
- [2]. M. Laavanya, V. Vijayaraghavan, Real time fake currency note detection using deep learning. *Int. J. Eng. Adv. Technol. (IJEAT)* 9(1S5) (2019). ISSN: 2249-8958
- [3]. Vidhate, Y. Shah, R. Biyani, H. Keshri, R. Nikhare, Fake currency detection application. *Int. Res. J. Eng. Technol. (IRJET)* 08(05) (2021). e-ISSN: 2395-0056.
- [4]. A. Mandankandy, K.E. Kannammal, Fake currency detection: a survey. *Gedrag en Organisatie* 33(4), 622–638 (2020).
- [5]. A. Upadhyaya, V. Shokeen, G. Srivastava, Analysis of counterfeit currency detection techniques for classification model (2018).
- [6]. T. D. Pham, C. Park, D. T. Nguyen, G. Batchuluun and K. R. Park, "Deep Learning-Based Fake-Banknote Detection for the Visually Impaired People Using Visible-Light Images Captured by Smartphone Cameras," in *IEEE Access*, vol. 8, pp. 63144-63161, 2020, doi: 10.1109/ACCESS.2020.2984019
- [7]. S. T. Kara, S. Loya, S. S. Raju, N. Vanteru and B. Rajulapati, "Detection of Fake Indian Currency Using Deep Convolutional Neural Network," *2023 IEEE 3rd Mysore Sub Section International Conference (MysuruCon)*, HASSAN, India, 2023, pp. 1-6, doi: 10.1109/MysuruCon59703.2023.10396993.
- [8]. B. Srilatha, C. H. Ithvika, P. Meenakshi, S. K. Pradhan, K. Pushpa Rani and M. Jayanth, "Fake Currency Detection Using Convolutional Neural Network and Image Processing," *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, Kamand, India, 2024, pp. 1-7, doi: 10.1109/ICCCNT61001.2024.10726269.
- [9]. M. Ghonge, T. Kachare, M. Sinha, S. Kakade, S. Nigade and S. Shinde, "Real Time Fake Note Detection using Deep Convolutional Neural Network," *2022 Second International Conference on Computer Science, Engineering and Applications (ICCSEA)*, Gunupur, India, 2022, pp. 1-6, doi: 10.1109/ICCSEA54677.2022.9936084.

- [10]. S. Mehta and A. Bhalla, "Fake Currency Detection Across Five Classes: A CNN-SVM Based Approach," *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, Kamand, India, 2024, pp. 1-6, doi: 10.1109/ICCCNT61001.2024.10725939.

CHAPTER 9

CERTIFICATES



INTERNSHIP COMPLETION CERTIFICATE



PROUDLY PRESENTED TO:
Anumula Abhinayasri

Student of Vignan's Nirula Institute of Technology and Science for Women
Reg No:..... 22NN1A1262 has successfully completed an
Internship on AI & ML Internship (13-05-2024 to 06-07-2024)
program at Vijayawada. During her internship program with
us, She was found punctual, hardworking and inquisitive.

This certificate was awarded by:

md. mobina
MOBINA MD

Managing Director



06-07-2024

Issue Date



CERTIFICATE OF TEAM MEMBER 1



INTERNSHIP COMPLETION CERTIFICATE



PROUDLY PRESENTED TO:
Mohammad Musarrat Nazia

Student of Vignan's Nirula Institute of Technology and Science for Women
Reg No:..... 22NN1A1288 has successfully completed an
Internship on AI & ML Internship (13-05-2024 to 06-07-2024)
program at Vijayawada. During her internship program with
us, She was found punctual, hardworking and inquisitive.

This certificate was awarded by:

md. mobina
MOBINA MD

Managing Director



06-07-2024

Issue Date



CERTIFICATE OF TEAM MEMBER 2



CERTIFICATE OF TEAM MEMBER 3



CERTIFICATE OF TEAM MEMBER 4